Climate Intelligence for Heatwave Monitoring and Early Warning

Cloud Computing Architecture, Containerization, Kubernetes Orchestration, and Resource Right-Sizing on Amazon Web Services

Mit Mhatre | Roll No.: 16014223052
Gurjit Banka | Roll No.: 16014223037
Kaustubh Patil | Roll No.: 16014223043

GitHub Repository: github.com/mitm2006/heat_forcast
Project Demonstration Video

October 2026

Abstract

Extreme heat is among the most consequential climate hazards affecting Indian cities, and timely, explainable information is central to reducing its impact. This report documents the Climate Intelligence platform, a heatwave monitoring and early warning system that analyses historical temperature records, produces seven-day forecasts with machine learning, and converts those forecasts into transparent heat-risk signals for five monitored cities: Pune, Mumbai, Nagpur, Delhi, and Ahmedabad. The principal focus of the report is the cloud computing foundation that makes the platform dependable and economical to operate. The system is packaged as Docker container images, orchestrated by a lightweight Kubernetes distribution (K3s), and hosted on an Amazon EC2 instance in the `us-east-1` region. A purpose-built terminal observability console compares reserved, limited, and actually consumed resources for every workload, and its findings drive a right-sizing plan that removes CPU overcommitment, reduces reserved memory by 59.4%, and lowers infrastructure cost by approximately 50% at the host level and 72.6% at the container level, while retaining high availability for the warning service.

1 Introduction

Heatwaves are prolonged periods of abnormally high temperature that place stress on public health, energy systems, agriculture, and outdoor labour. In India, the India Meteorological Department (IMD) issues official heatwave bulletins based on station observations, departures from normal, duration, and regional coverage. Complementary decision-support tools that translate forecast temperatures into clear, explainable risk signals can help city administrations, health agencies, and researchers prepare earlier and communicate more effectively.

The Climate Intelligence platform is built for this purpose. It combines three analytical capabilities:

- **Monitoring:** analysis of daily maximum and minimum temperature, humidity, rainfall, and seasonal behaviour against a smoothed climatological normal.

- **Prediction:** Random Forest regression models that forecast daily maximum and minimum temperature for each monitored city.
- **Early warning:** a transparent, rule-based heat-risk classification and a 0–100 prioritisation score that are presented with a plain-language explanation.

An early warning service has demanding operational requirements. It must be available when a hazard approaches, it must serve many concurrent readers during an event, it must be updated without interruption as new weather data arrives, and it must be affordable enough to run continuously. These requirements are the reason the platform is designed around cloud computing principles rather than as a single-machine application. This report therefore treats the cloud architecture as the primary subject, while keeping the heatwave analytics visible as the workload that the infrastructure exists to serve.

1.1 Objectives

The objectives of the cloud engineering effort are as follows.

1. Package the analytical engine and the web interface as portable, immutable container images.
2. Run the packages on a Kubernetes cluster that provides self-healing, rolling updates, service discovery, and horizontal scaling.
3. Host the cluster on Amazon Web Services using an appropriately sized virtual machine, with network access controlled by security groups.
4. Provide continuous visibility into resource allocation and consumption without the overhead of a heavyweight monitoring stack.
5. Use the observed data to right-size the workload, eliminating waste while preserving availability and responsiveness.

1.2 Course Experiments Used in the Project

The project combines concepts and practical techniques covered in the following cloud computing experiments.

Experiment No.	Experiment Name
Exp 2	Containerization, Docker and Docker Hub
Exp 3	Application deployment using Kubernetes (PaaS)
Exp 4	Windows and Linux Virtual Machine Instance (IaaS)
Exp 7	Cloud cost optimization with resource right-sizing

Table 1: Course experiments applied while developing and deploying the Climate Intelligence project.

1.3 Scope and Structure

Section 2 summarises the heatwave analytics that form the workload. Section 3 presents the overall cloud topology. Sections 4 to 6 describe containerization, orchestration, and the AWS

infrastructure. Section 7 covers observability, Section 8 analyses resource utilisation, and Section 9 presents the right-sizing plan and its financial impact. Sections 10 to 12 discuss reliability, security, and the evolution path, followed by limitations and conclusions.

2 Heatwave Monitoring and Early Warning Workload

2.1 Data Sources

Historical daily weather records and short-range forecasts are obtained from the Open-Meteo Historical Weather and Forecast APIs. The pipeline downloads these records for every configured city and stores them as structured files. If the external service is unavailable for a particular city, the pipeline retains the existing local data for that city or falls back to the machine learning forecast, so that the platform continues to operate. Each record carries a data source label and each forecast carries a forecast source label, and the dashboard presents a banner so that users always know the provenance of the numbers displayed. City definitions, including latitude, longitude, state, and region type (plains or coastal), are held in a single configuration module, which makes adding a new location a configuration change rather than a code change.

2.2 Forecasting Models

One Random Forest regression model is trained for maximum temperature and one for minimum temperature for each city. Only information from earlier days is used as input, which prevents leakage of future values into the model. The feature set is summarised in Table 2.

Feature group	Description
Maximum temperature lags	Values from 1, 2, 3, 7, and 14 days earlier
Minimum temperature lags	Values from 1 and 7 days earlier
Rolling means	Averages over the previous 3, 7, and 14 days
Seasonality	Annual and monthly cyclical encodings of the calendar date

Table 2: Input features used by the temperature forecasting models.

Model quality is assessed with a chronological hold-out: earlier records are used for fitting and later records for evaluation, which mirrors how the model is used operationally. Mean absolute error (MAE), root mean squared error (RMSE), and the coefficient of determination (R^2) are recorded for every city. The dashboard reports a forecast accuracy indicator of 0.89°C mean absolute error against historical observations. Trained models are serialised with Joblib and loaded into memory when the API container starts, so that inference is a read-only, in-memory operation.

2.3 Heat-Risk Classification

For each forecast date the system computes the temperature anomaly relative to a smoothed day-of-year normal:

$$\Delta T_d = T_{\text{max},d}^{\text{forecast}} - T_{\text{normal},d}. \tag{1}$$

The risk level is then determined by a transparent heuristic that uses the absolute forecast maximum temperature, the positive departure from normal, and the region type. Plains and coastal regions are treated differently, consistent with the way IMD criteria distinguish them, and IMD-referenced temperature thresholds, such as the 40°C heatwave and 45°C severe reference lines drawn on the dashboard charts, are used for comparison. The exact rules are contained in a single function so that they can be reviewed and adjusted.

A complementary 0–100 risk score combines forecast temperature, positive anomaly, and relative humidity. This score is a project-specific prioritisation aid. It is not the official IMD severity index and it is not a clinical heat-stress index.

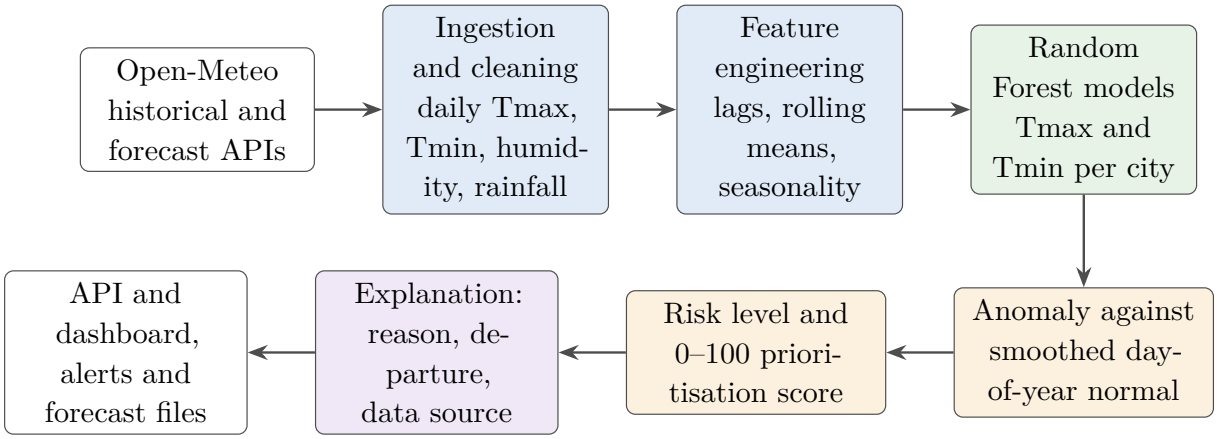


Figure 1: End-to-end analytics pipeline from weather data to explainable heat-risk output.

2.4 Application Interfaces

The analytical layer is exposed through a RESTful API and an interactive web dashboard. The API provides summary, forecast, and alert endpoints together with a health endpoint used by the orchestration layer. The dashboard provides a city selector, temperature unit selection, selectable historical windows, a command centre with observed temperature trends against the climatological normal, a seven-day forecast risk outlook, a regional comparison of forecast peaks, a geospatial risk map, and model quality information. Figure 2 shows the dashboard served by the cluster.

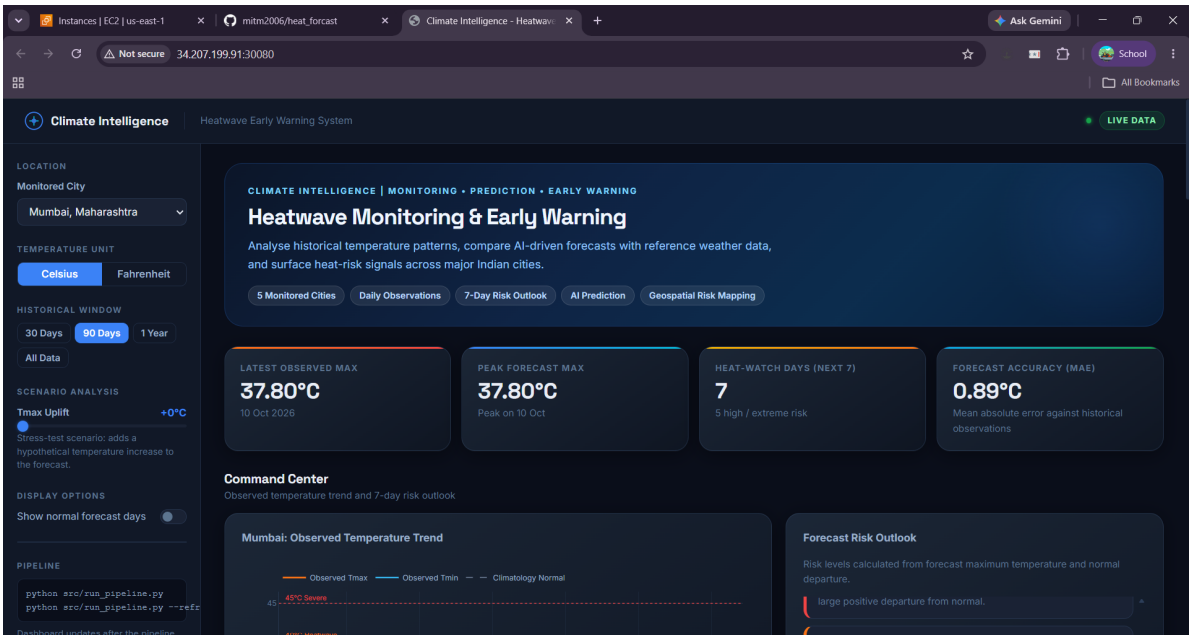


Figure 2: Heatwave Monitoring and Early Warning dashboard served from the Kubernetes NodePort on the cloud host, showing the monitored city selector, observed maximum temperature, peak forecast maximum, forecast risk days, and forecast accuracy indicators.

2.5 Responsible Use

The platform produces modelled heat-risk signals. These are not official warnings. Official heatwave declarations depend on station-specific observations, event duration, regional coverage, and operational assessment, and the platform's outputs must not be described as IMD alerts. Users should rely on IMD bulletins and local authority guidance for decisions that affect public safety.

3 Cloud Architecture Overview

3.1 Why Cloud Computing for Early Warning

The characteristics of an early warning service map directly to well-established cloud capabilities, as summarised in Table 3.

Operational need	Cloud capability applied
Availability during hazardous events	Multiple replicas behind a service, health probes, automatic restart of failed containers
Safe delivery of model and data updates	Immutable container images and rolling updates with no unavailable pods
Variable reader demand	Horizontal scaling of stateless frontend replicas
Reproducible deployments	Declarative manifests and container build files held in version control
Controlled public exposure	Security groups, ingress routing, and internal-only services for the API
Sustainable running cost	Right-sizing of the virtual machine and of container resource reservations
Operational visibility	Metrics from the Kubernetes Metrics Server and the host operating system, presented in a terminal console

Table 3: Mapping of early warning requirements to cloud capabilities.

3.2 Logical Topology

Figure 3 shows the logical topology of the deployment. Browser traffic enters the AWS-hosted virtual machine, where the K3s ingress controller and node port expose the frontend service. The frontend forwards API calls over the cluster network to the backend service, which is reachable only from inside the cluster. The backend loads trained models and processed data and synchronises weather data from Open-Meteo. The observability console runs on the same host and reads metrics from the cluster, the operating system, and the EC2 instance metadata service.

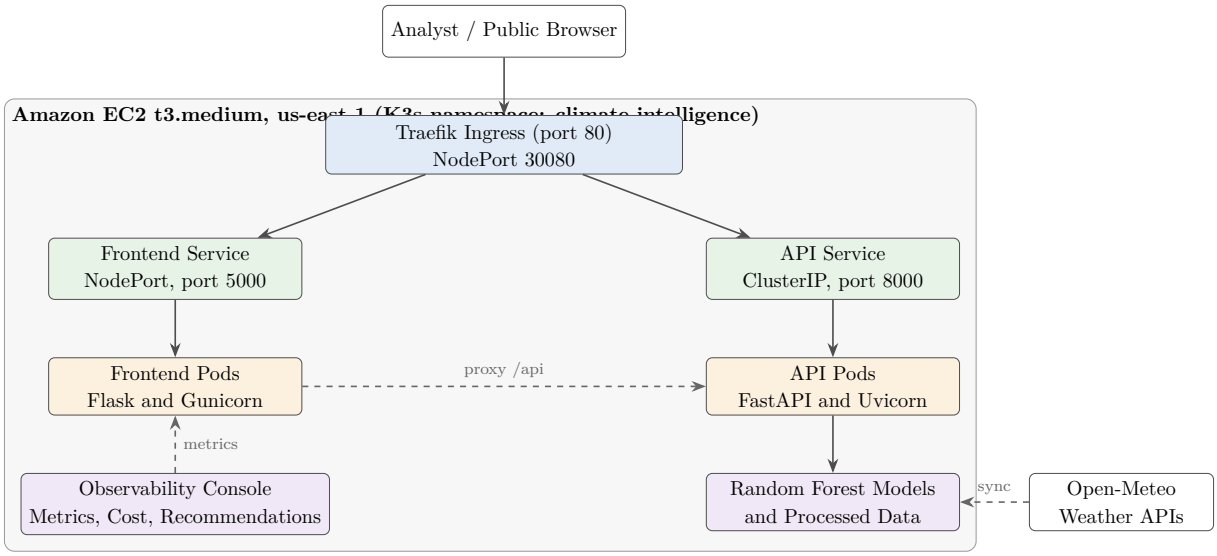


Figure 3: Logical cloud topology of the Climate Intelligence platform.

3.3 Technology Stack

Table 4 lists the technologies used at each layer and the role each plays in the cloud deployment.

Layer	Technologies	Cloud role
Analytics and ML	Python 3.11, Scikit-learn, Pandas, NumPy, Joblib	Stateless inference using models loaded in memory at pod start-up
Backend API	FastAPI, Uvicorn, Pydantic	Microservice exposing HTTP endpoints and health probes for the orchestrator
Frontend	Flask, Jinja2, Gunicorn, Leaflet.js, Plotly.js	Decoupled web tier that proxies API requests over the cluster network
Containers	Docker, containerd	Immutable packaging of application and dependencies
Orchestration	K3s, Traefik, CoreDNS, Metrics Server	Scheduling, self-healing, service discovery, routing, and metrics
Infrastructure	AWS EC2, VPC, Security Groups, EBS gp3	Virtual compute, network isolation, and block storage
Observability	Python `rich`, IMDSv2, `kubectl top`	Resource audit, right-sizing recommendations, and cost modelling

Table 4: Technology stack and cloud role of each layer.

4 Containerization with Docker

Containerization is the unit of deployment for the platform. It encapsulates the Python runtime, libraries, application code, trained models, and data into an image that behaves identically on a developer workstation and on the cloud host.

4.1 Backend Image

The backend image is built from `python:3.11-slim`. This base was chosen over Alpine because scientific Python packages such as NumPy and Scikit-learn are distributed as pre-compiled manylinux wheels, which install directly on Debian-based slim images. Alpine would require compiling these packages from source, increasing build time and image size. The resulting backend image is approximately 380 MB.

Key design decisions in the backend image include:

- **Least privilege:** a dedicated unprivileged user (`appuser`, UID 1000) runs the process, which reduces the impact of a container compromise.
- **Layer ordering:** dependencies are installed before the application source is copied, so routine code changes reuse the cached dependency layer and rebuild quickly.
- **Built-in health probe:** a health check queries the `/health` endpoint every 15 seconds so that the container engine can detect a stalled process.
- **Explicit binding:** Uvicorn listens on all interfaces within the container network namespace on port 8000.

Listing 1: Structure of the backend container build file (abridged).

```
FROM python:3.11-slim
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1 PIP_NO_CACHE_DIR=1
```

```
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ dashboard/ data/ models/ outputs/ /app/
RUN useradd -u 1000 -m appuser && chown -R appuser:appuser /app
USER appuser
EXPOSE 8000
HEALTHCHECK --interval=15s --timeout=5s --retries=3 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')
    "
CMD ["python", "-m", "uvicorn", "dashboard.api:app", "--host", "0.0.0.0", "--port", "8000"]
```

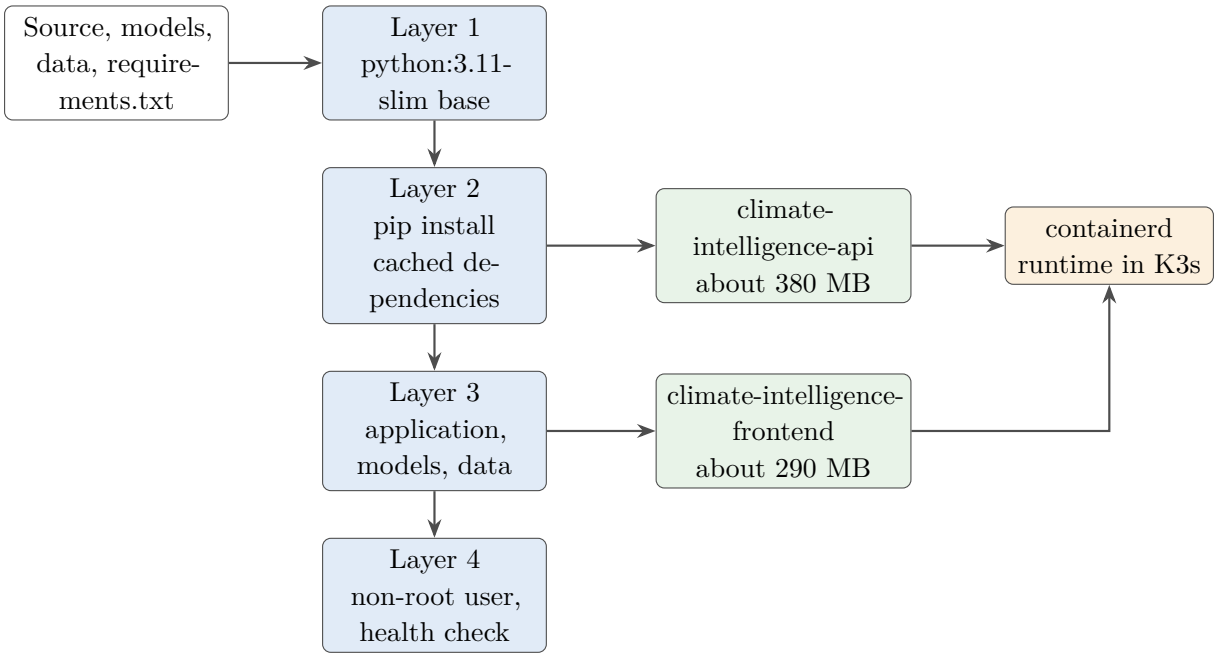


Figure 4: Container image build pipeline. Dependencies are cached in an early layer so code changes rebuild quickly.

4.2 Frontend Image

The frontend image packages the Flask web application, the Leaflet and Plotly templates, and a reverse proxy route for API calls. Its environment defaults set the listening address and port (0.0.0.0:5000) and the backend address. It exposes a health endpoint on port 5000 for probes. The frontend image is approximately 290 MB.

4.3 Multi-Container Composition

Docker Compose describes the two services and the network that joins them. A dedicated bridge network provides built-in DNS, so the frontend resolves the backend by service name. The data directory is mounted read-only, which guarantees that running containers cannot alter the underlying datasets. A health-conditioned dependency ensures that the frontend does not accept traffic until the backend has reported healthy. The same logical structure is carried over into the Kubernetes manifests.

5 Cluster Orchestration with Kubernetes

5.1 The K3s Distribution

The platform uses K3s, a certified lightweight Kubernetes distribution suited to single-node cloud hosts. K3s replaces etcd with an embedded datastore, which lowers control-plane memory use, and bundles the components needed for a complete cluster: the containerd runtime, the Traefik ingress controller, CoreDNS for in-cluster name resolution, and the Kubernetes Metrics Server for resource telemetry. This allows a full orchestration environment to run on a two-vCPU virtual machine.

5.2 Declarative Manifests

All cluster resources are defined as manifests and applied with `kubectl`. They are summarised in Table 5.

Resource	**Purpose**
Namespace `climate-intelligence`	Isolates all project workloads and allows namespace-scoped quotas
ConfigMap `climate-config`	Holds runtime settings such as time zone (Asia/Kolkata), API address, and debug flag, so configuration changes do not require new images
Deployment `climate-api`	Two replicas of the backend with probes, resource requests, and limits
Deployment `climate-frontend`	Frontend replicas with probes, resource requests, and limits
Service `climate-api-service`	ClusterIP on port 8000, reachable only inside the cluster
Service `climate-frontend-service`	NodePort 30080 mapped to frontend port 5000
Ingress `climate-ingress`	Path-based routing: `/api` to the backend and `/` to the frontend

Table 5: Kubernetes resources that define the deployment.

Figure 5 shows the namespace after deployment: two backend pods and two frontend pods in the Running state, the two services, the two deployments, and their ReplicaSets.

```
(.venv) ubuntu@ip-172-31-34-240:~/climate_intelligence_heatwave$ sudo k3s kubectl get all -n climate-intelligence
NAME                                     READY   STATUS    RESTARTS   AGE
pod/climate-api-8c56cf8d9-6gwg6          1/1     Running   0          7m36s
pod/climate-api-8c56cf8d9-9g8tq          1/1     Running   0          7m36s
pod/climate-frontend-cdd787c6d-96xqf     1/1     Running   0          7m36s
pod/climate-frontend-cdd787c6d-wg8ch     1/1     Running   0          7m36s

NAME                               TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)          AGE
service/climate-api-service        ClusterIP   10.43.57.240   <none>        8000/TCP         7m36s
service/climate-frontend-service   NodePort    10.43.54.11    <none>        5000:30080/TCP   7m36s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/climate-api         2/2     2            2           7m36s
deployment.apps/climate-frontend    2/2     2            2           7m36s

NAME                                           DESIRED   CURRENT   READY   AGE
replicaset.apps/climate-api-8c56cf8d9          2         2         2       7m36s
replicaset.apps/climate-frontend-cdd787c6d     2         2         2       7m36s
(.venv) ubuntu@ip-172-31-34-240:~/climate_intelligence_heatwave$
```

Figure 5: Workloads in the `climate-intelligence` namespace: pods, services, deployments, and ReplicaSets.

5.3 Rolling Updates and Probes

Both deployments use a rolling update strategy with `maxSurge: 1` and `maxUnavailable: 0`. During an update, Kubernetes starts one new pod, waits until it is ready, and only then retires an old pod. Service capacity never drops below the desired replica count, which is important for a warning service that must remain reachable while new data or models are released.

Two probes govern container lifecycle. The liveness probe verifies that the process is responsive and causes the kubelet to restart the container after repeated failures. The readiness probe verifies that models and data are loaded before the pod is added to the service endpoints, so that clients are never routed to a pod that is still initialising.

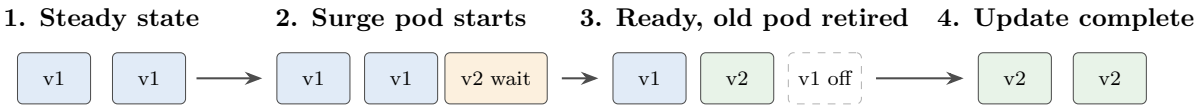


Figure 6: Rolling update sequence used for both deployments.

5.4 Horizontal Scaling and Self-Healing

Because both tiers are stateless, replica counts can be changed at any time without data migration. Figure 7 shows the frontend deployment being scaled from two to four replicas with a single command; the additional pods reached the Running state within seconds.

```
(.venv) ubuntu@ip-172-31-34-240:~/climate_intelligence_heatwave$ sudo k3s kubectl scale deployment/climate-frontend -n climate-intelligence --re
plicas=4
deployment.apps/climate-frontend scaled
(.venv) ubuntu@ip-172-31-34-240:~/climate_intelligence_heatwave$ sudo k3s kubectl get pods -n climate-intelligence
NAME                                READY   STATUS    RESTARTS   AGE
climate-api-8c56cf8d9-6gwg6         1/1     Running   0          8m26s
climate-api-8c56cf8d9-9g8tq         1/1     Running   0          8m26s
climate-frontend-cdd787c6d-96xqf    1/1     Running   0          8m26s
climate-frontend-cdd787c6d-kvj8r    1/1     Running   0          14s
climate-frontend-cdd787c6d-wg8ch    1/1     Running   0          8m26s
climate-frontend-cdd787c6d-xkzs4    1/1     Running   0          14s
(.venv) ubuntu@ip-172-31-34-240:~/climate_intelligence_heatwave$
```

Figure 7: Horizontal scaling of the frontend deployment from two to four replicas.

Self-healing is provided by the ReplicaSet controllers. When a backend pod is removed, the controller immediately creates a replacement to restore the desired count, while the remaining healthy replica continues to serve traffic. Figure 8 shows the replacement pods starting; they remain out of the service endpoints (`0/1` ready) until the readiness probe confirms that models have been loaded.

```
(.venv) ubuntu@ip-172-31-34-240:~/climate_intelligence_heatwave$ sudo k3s kubectl delete pod -l app=climate-api -n climate-intelligence | head -
n 1
pod "climate-api-8c56cf8d9-6gwg6" deleted from climate-intelligence namespace
(.venv) ubuntu@ip-172-31-34-240:~/climate_intelligence_heatwave$ sudo k3s kubectl get pods -n climate-intelligence
NAME                                READY   STATUS    RESTARTS   AGE
climate-api-8c56cf8d9-6fljl         0/1     Running   0          9s
climate-api-8c56cf8d9-jnfz9         0/1     Running   0          9s
climate-frontend-cdd787c6d-96xqf    1/1     Running   0          9m20s
climate-frontend-cdd787c6d-kvj8r    1/1     Running   0          68s
climate-frontend-cdd787c6d-wg8ch    1/1     Running   0          9m20s
climate-frontend-cdd787c6d-xkzs4    1/1     Running   0          68s
(.venv) ubuntu@ip-172-31-34-240:~/climate_intelligence_heatwave$
```

Figure 8: Automatic replacement of backend pods by the ReplicaSet controller, with readiness gating before traffic is admitted.

5.5 Requests, Limits, and Overcommitment

Kubernetes governs compute capacity with two values per container. A *request* is the amount the scheduler reserves for the container when choosing a node. A *limit* is the ceiling enforced by the Linux kernel. CPU limits are enforced through Completely Fair Scheduler bandwidth quotas, so a container that exceeds its limit is throttled. Memory limits are enforced through control groups, so a container that exceeds its limit is terminated by the out-of-memory killer (exit code 137).

The node is considered *overcommitted* when the sum of limits exceeds allocatable capacity:

$$\text{Overcommit ratio} = \frac{\sum_i L_i}{C_{\text{node}}}. \tag{2}$$

The initial manifests assigned CPU limits of 500m to each of the two backend pods and 400m to each of the four frontend pods, giving $2 \times 500 + 4 \times 400 = 2600$m on a node with $C_{\text{node}} = 2000$m, an overcommit ratio of 130%. If every pod were to burst at once, the kernel would throttle CPU time and response latency would rise. This finding motivates the right-sizing work in Section 9.

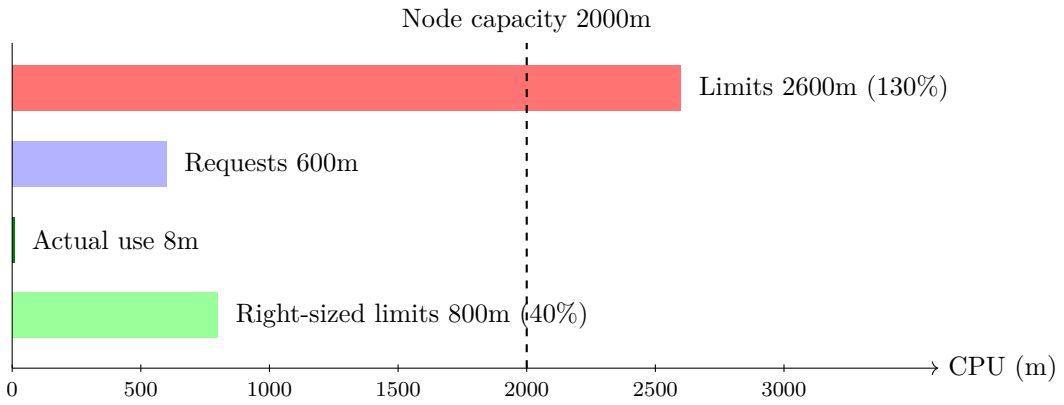


Figure 9: CPU reservations and ceilings for the application workload against node capacity. Current limits exceed the node by 30%.

6 Cloud Infrastructure on Amazon Web Services

6.1 Compute and Storage

The platform runs on a single Amazon EC2 instance in the `us-east-1` region. The configuration is given in Table 6.

Attribute	Value
Instance type	`t3.medium`, burstable general purpose, AWS Nitro System
Processor	2 vCPUs, Intel Xeon Platinum 8259CL at 2.50 GHz
Memory	4.0 GiB (3834 MiB visible to the operating system)
Operating system	Ubuntu 24.04 LTS
Root storage	20 GiB Amazon EBS gp3, 3000 baseline IOPS, 125 MB/s throughput
Orchestrator	K3s with containerd

Table 6: Configuration of the cloud host.

The T3 family accrues CPU credits during idle periods and spends them during bursts. This suits the platform well: observed CPU use is a few percent of capacity most of the time, with short bursts when users load the dashboard or when the pipeline refreshes data.

6.2 Network Controls

The instance resides in the default Virtual Private Cloud (CIDR 172.31.0.0/16) and is protected by a dedicated security group. Port 22 provides administrative SSH access, port 80 serves the ingress controller, and port 30080 exposes the frontend NodePort. Ports 5000 and 8000 exist only for direct diagnostics and should be restricted to trusted source addresses or closed in a hardened production configuration. The API service is of type ClusterIP, so it is not reachable from outside the cluster except through the ingress path.

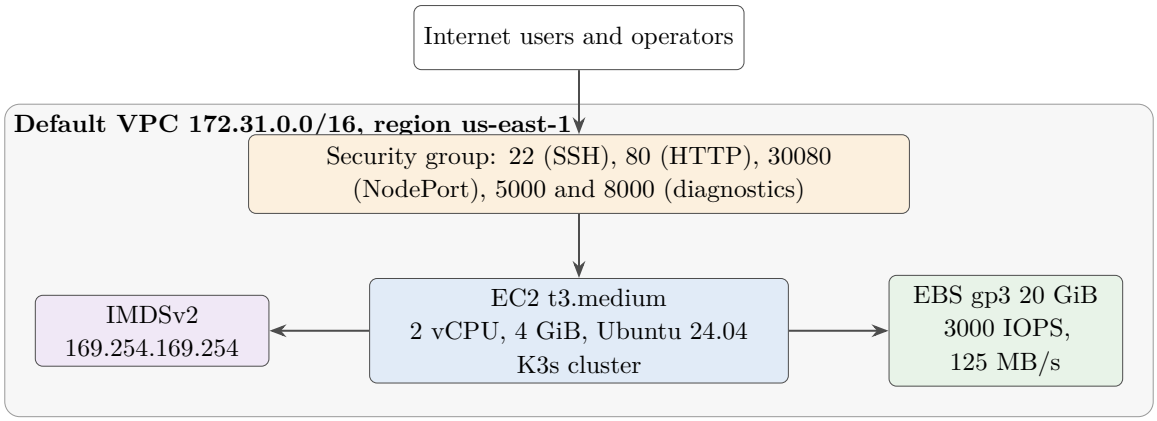


Figure 10: AWS network and compute layout of the deployment.

6.3 Instance Metadata Service

The observability console discovers the host profile, including instance type and availability zone, by querying the EC2 Instance Metadata Service using IMDSv2. IMDSv2 requires a short-lived session token obtained with an HTTP PUT request, which protects the metadata endpoint against server-side request forgery attacks that affect the older IMDSv1 model.

Listing 2: IMDSv2 token retrieval and instance type query.

```
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/instance-type
```

7 Observability and Resource Monitoring

7.1 Design Rationale

A full Prometheus and Grafana stack can consume well over a gibibyte of memory, which is disproportionate on a small cloud instance whose entire workload uses under half a gibibyte. The platform therefore includes a lightweight, non-invasive terminal console that reads data already available on the host: Linux `/proc` statistics, the Kubernetes Metrics Server, the Kubernetes API for requests and limits, and the EC2 metadata service. It requires no additional agents in the cluster and does not alter any workload.

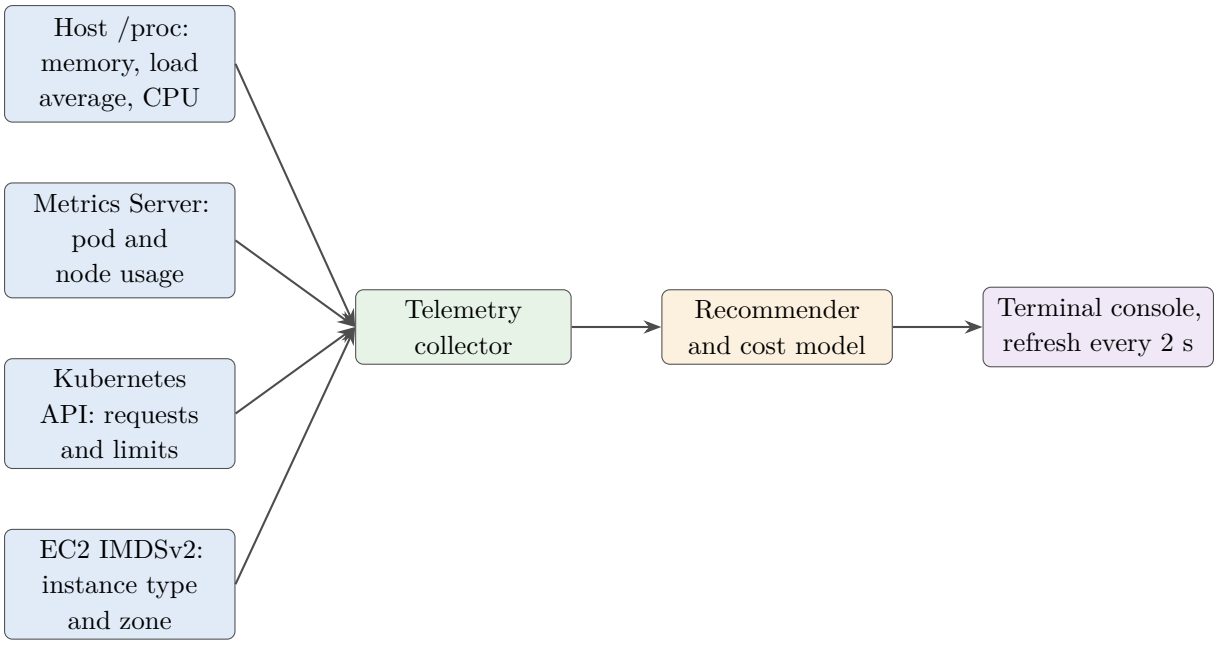


Figure 11: Data flow of the observability console. No agents are installed in the cluster.

7.2 Console Capabilities

The console is built with the Python `rich` library and refreshes values in place every two seconds without clearing the screen. Keyboard hotkeys switch between views. The views are listed in Table 7.

View	Content
Utilization	Host memory and load averages; node capacity against total requests and limits; pod-level actual usage, requests, and limits with a per-pod diagnosis
Recommendations	Root causes of inefficiency and the proposed replica counts, requests, and limits
Cost model	Hourly, monthly, and annual cost of the current and proposed host and container configurations
Manifests	Ready-to-apply `kubectl` commands and right-sized YAML for zero-downtime remediation

Table 7: Views provided by the observability console.

Figure 12 shows the utilisation view of the console, which compares total capacity, reserved requests, limits, and measured usage for CPU and memory, and breaks the namespace down pod by pod.

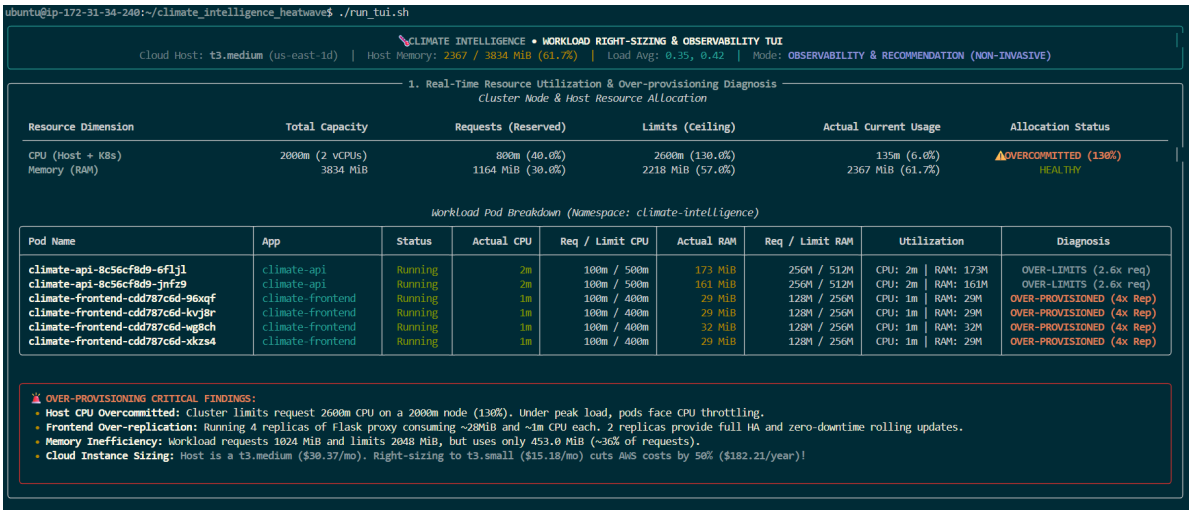


Figure 12: Resource utilisation and over-provisioning diagnosis in the observability console.

8 Resource Utilisation Analysis

8.1 Host-Level Findings

Table 8 summarises host-level allocation and use for the `t3.medium` instance as reported by the console.

Resource	Capacity	Requests	Limits	Actual use
CPU	2000m (2 vCPU)	800m (40.0%)	2600m (130.0%)	135m (6.0%)
Memory	3834 MiB	1164 MiB (30.0%)	2218 MiB (57.0%)	2367 MiB (61.7%)

Table 8: Host-level allocation and usage, including K3s system components. The one- and five-minute load averages were 0.35 and 0.42.

CPU limits at 130% of capacity are the principal risk. Actual CPU use is only 6%, so no throttling occurs under current traffic, but the configuration provides no guarantee under a sudden burst. Host memory use is dominated by the operating system and the K3s control plane rather than by the application pods.

8.2 Workload-Level Findings

Table 9 lists measured usage for the six application pods compared with their reservations.

Pod	CPU use	CPU req/lim	RAM use	RAM req/lim
climate-api (replica 1)	2m	100m / 500m	173 MiB	256 / 512 MiB
climate-api (replica 2)	2m	100m / 500m	161 MiB	256 / 512 MiB
climate-frontend (replica 1)	1m	100m / 400m	29 MiB	128 / 256 MiB
climate-frontend (replica 2)	1m	100m / 400m	29 MiB	128 / 256 MiB
climate-frontend (replica 3)	1m	100m / 400m	32 MiB	128 / 256 MiB
climate-frontend (replica 4)	1m	100m / 400m	29 MiB	128 / 256 MiB

Table 9: Measured pod usage against requests and limits.

Three conclusions follow from these figures.

1. **The frontend tier is heavily over-provisioned.** Each frontend replica uses about 1m of CPU and 30 MiB of memory, roughly 1% and 23% of the respective requests. Four replicas of a thin proxy are unnecessary; two replicas already provide redundancy and zero-downtime updates.
2. **The backend is memory-bound, not CPU-bound.** Each API pod holds trained models and data in memory and therefore uses 160 to 175 MiB, while CPU use is about 2m. Backend memory reservations are reasonable and should be retained.
3. **Aggregate reservations exceed use by a wide margin.** Workload pods use 453 MiB of memory against 1024 MiB requested (about 44%), and 8m of CPU against 600m requested (about 1.3%).

9 Right-Sizing and Cost Optimisation

9.1 Recommended Workload Configuration

The recommendation retains two backend replicas for availability, reduces the frontend to two replicas, and sets requests and limits close to measured behaviour with sensible headroom. The resulting changes are shown in Table 10.

Metric	Current	Recommended	Change
Frontend replicas	4	2	−33.3% pod count
Backend replicas	2	2	No change; high availability retained
Total CPU requests	600m	150m	−75.0%
Total CPU limits	2600m (130% of node)	800m (40% of node)	−69.2%; overcommitment eliminated
Total memory requests	1024 MiB	416 MiB	−59.4%
Total memory limits	2048 MiB	704 MiB	−65.6%

Table 10: Current and recommended workload resource configuration.

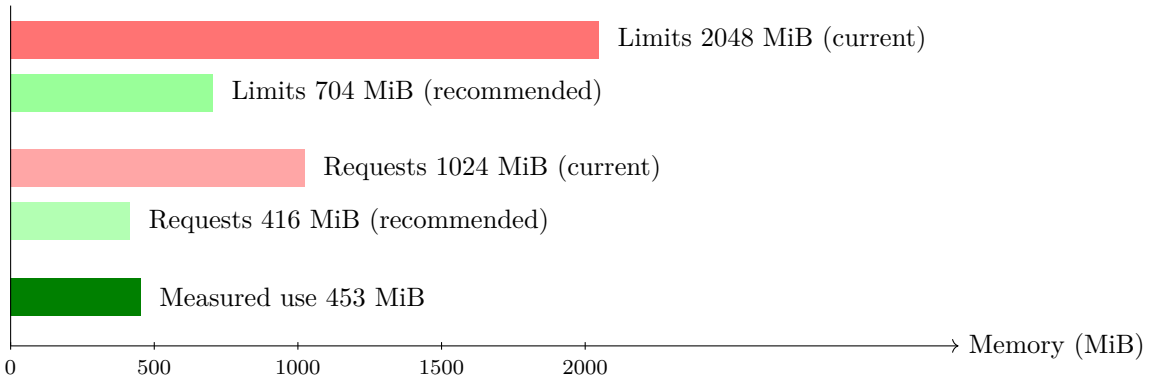


Figure 13: Workload memory reservations, limits, and measured use before and after right-sizing.

Bringing total CPU limits to 800m, which is 40% of node capacity, removes CFS quota contention. Throttling can no longer occur because the sum of ceilings is below the physical capacity, and the scheduler retains ample room for system components and for future workloads.

9.2 Host Instance Right-Sizing

Once the workload reservations are corrected, the host itself can be reduced from `t3.medium` to `t3.small`, which has the same two vCPUs but 2.0 GiB of memory. The memory budget for the smaller instance is shown in Table 11.

Component	Approximate memory
Ubuntu and background services	380 MiB
K3s control plane and containerd	700 MiB
Traefik ingress and CoreDNS	150 MiB
Right-sized workload (2 backend and 2 frontend pods)	390 MiB
Estimated peak footprint	**about 1.62 GiB**
Remaining headroom on a 2.0 GiB instance	about 380 MiB

Table 11: Estimated memory budget on a `t3.small` host.

The headroom is adequate for steady operation. Because it is a moderate margin, the migration should be carried out in a phased manner with the observability console monitoring memory pressure after the change.

9.3 Financial Impact

Costs are computed from AWS on-demand rates for the `us-east-1` region. Two deployment models are considered.

Scenario A: EC2 host migration. A `t3.medium` costs $0.0416 per hour, or about $30.37 per month. A `t3.small` costs $0.0208 per hour, or about $15.18 per month. The saving is about $15.18 per month, or $182.21 per year, a 50% reduction.

Scenario B: Container-level billing. If the workload were run on a serverless container platform such as Amazon EKS with AWS Fargate, billing would be proportional to reserved resources at $0.04048 per vCPU-hour and $0.004445 per GB-hour. The current reservations (0.6 vCPU and 1.0 GB) correspond to about $20.98 per month. The right-sized reservations (0.15 vCPU and 0.41 GB) correspond to about $5.75 per month, a saving of $15.22 per month or 72.6%.

Deployment model	Current	Right-sized	Saving
EC2 host (per month)	$30.37	$15.18	50.0%
Container reservations on Fargate (per month)	$20.98	$5.75	72.6%

Table 12: Monthly cost comparison before and after right-sizing.

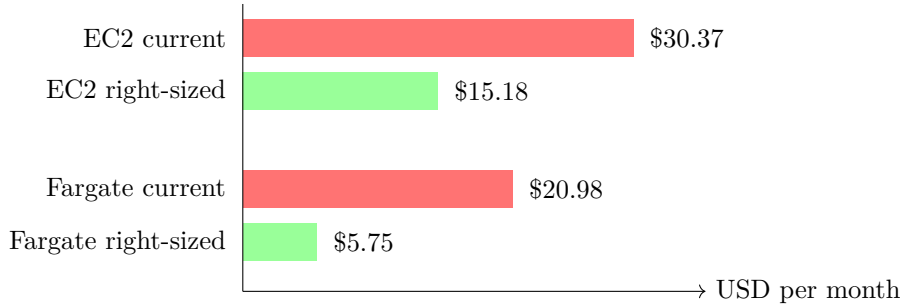


Figure 14: Monthly cost before and after right-sizing for the two deployment models.

These savings accumulate for as long as the early warning service runs, and they free budget for additional cities, more frequent data refreshes, or redundancy.

9.4 Rollout Procedure

Right-sizing is applied without downtime. The recommended manifests are applied with `kubectl apply`, and the rolling update strategy replaces pods one at a time while readiness probes keep traffic on healthy replicas. The host instance change is then performed during a low-demand window by stopping the instance, changing the instance type, and starting it again, after which K3s restores the namespace from its persisted state.

10 Reliability and Availability

Reliability for an early warning service is judged by whether users can reach correct information when it matters most. The design contributes in several ways.

- **Redundancy:** two replicas of each tier mean that the loss of one pod does not interrupt service.
- **Automatic recovery:** liveness probes and ReplicaSet controllers restore failed or removed pods without operator action.

- **Traffic safety:** readiness probes keep uninitialised pods out of service, and rolling updates with zero unavailable pods protect users during releases.
- **Statelessness:** no session data is held in container memory, so any pod can be replaced without loss of user state.
- **Graceful degradation of data:** when the external weather API is unavailable, the pipeline retains existing local data or switches to the machine learning forecast, and the dashboard labels the source so that users are not misled.

A single EC2 instance remains a single point of failure at the infrastructure level. Section 12 describes how multi-node and multi-zone deployments remove this limitation.

11 Security Considerations

- **Container hardening:** application processes run as an unprivileged user, and slim base images reduce the attack surface.
- **Network exposure:** the backend is a ClusterIP service, and only the ingress and frontend ports are intended for public access. Diagnostic ports should be restricted by source address or closed.
- **Metadata protection:** IMDSv2 is used for instance metadata access.
- **Read-only data:** datasets are mounted read-only in the container composition so that running services cannot modify them.
- **Configuration separation:** runtime settings are kept in a ConfigMap rather than in images. Any future credentials should be stored as Kubernetes Secrets or in AWS Secrets Manager.
- **Transport security:** the current endpoint is served over HTTP. A production deployment should terminate TLS at the ingress using certificates from AWS Certificate Manager or Let's Encrypt.

12 Evolution Path

The present deployment is a compact single-node cluster. The same container images and manifests can be carried to larger managed environments as demand grows. Table 13 outlines a staged path.

Capability	Proposed cloud service or practice
Automatic scaling	Kubernetes Horizontal Pod Autoscaler on CPU or request rate for the frontend and backend
Multi-zone availability	Managed Kubernetes (Amazon EKS) with node groups across multiple availability zones
Image distribution	Amazon Elastic Container Registry with automated image builds in a CI/CD pipeline
Data and model storage	Amazon S3 for historical data, model artefacts, and generated outputs, with versioning
Scheduled data refresh	Kubernetes CronJob or Amazon EventBridge to run the pipeline and refresh forecasts on a schedule
Alert distribution	Amazon SNS for notification of non-green modelled risk levels to subscribed recipients
Managed monitoring	Amazon CloudWatch or Prometheus with alert rules for resource pressure and error rates
Edge delivery and protection	Application Load Balancer, Amazon CloudFront, and AWS WAF in front of the ingress
Cost governance	Namespace resource quotas, AWS Budgets, and periodic right-sizing reviews

Table 13: Staged evolution of the platform on AWS.

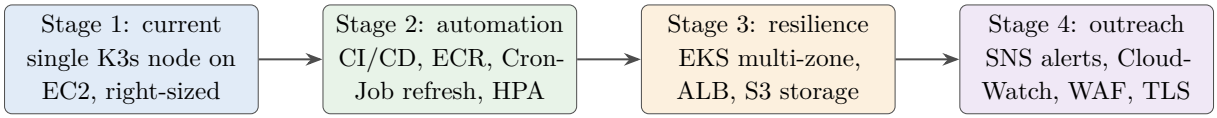


Figure 15: Staged evolution of the platform.

The scheduled refresh and notification capabilities are especially relevant to heatwave early warning, since they convert the forecasting pipeline from an on-demand tool into a continuously operating service that can reach the people who need to act.

13 Limitations

- The platform produces modelled heat-risk signals based on forecast maximum temperature, departure from normal, and region type. These do not replace IMD bulletins and are not official alerts.
- The 0–100 risk score is a project-specific prioritisation measure and is not a clinical heat-stress index.
- The Random Forest models use temperature history and seasonality only. Additional predictors such as humidity, wind, soil moisture, and large-scale circulation indices could improve skill.
- The current deployment uses one EC2 host, so infrastructure-level redundancy is not yet available.
- The right-sizing recommendations rely on observed utilisation under present traffic. They should be reviewed periodically, particularly before expected periods of high public interest.

14 Conclusion

The Climate Intelligence platform combines temperature analytics, machine learning forecasts, and explainable heat-risk classification in a service designed for the reliability that early warning demands. Its cloud foundation is deliberately lean and complete. Docker images make the workload portable and reproducible. K3s provides self-healing, readiness-gated traffic, zero-downtime updates, and horizontal scaling on a modest virtual machine. AWS supplies the compute, storage, and network controls on which the cluster runs. A purpose-built observability console reveals how reserved resources compare with actual use.

That visibility exposed a significant inefficiency: CPU limits at 130% of node capacity, four idle frontend replicas, and memory reservations more than double actual use. The recommended changes remove CPU overcommitment, cut reserved memory by 59.4%, and enable a move to a smaller host that halves compute cost, while two replicas of each tier preserve high availability. The staged evolution path then offers a clear route toward multi-zone resilience, automatic scaling, scheduled forecasts, and direct alert distribution as the system grows from a project prototype toward a dependable regional decision-support service.

References

[1] Open-Meteo. Historical Weather API documentation. https://open-meteo.com/en/docs/historical-weather-api

[2] Open-Meteo. Weather Forecast API documentation. https://open-meteo.com/en/docs

[3] India Meteorological Department. Heat wave criteria and bulletins. https://mausam.imd.gov.in/pdfs/heatcolduser/morning_heat_bulletin.pdf

[4] The Kubernetes Authors. Kubernetes documentation: Managing resources for containers; Configure liveness, readiness and startup probes. https://kubernetes.io/docs/

[5] K3s Project. K3s: Lightweight Kubernetes documentation. https://docs.k3s.io/

[6] Docker Inc. Dockerfile reference and Docker Compose documentation. https://docs.docker.com/

[7] Amazon Web Services. Amazon EC2 instance types, EC2 on-demand pricing, and Instance Metadata Service Version 2 documentation. https://docs.aws.amazon.com/ec2/

[8] Amazon Web Services. AWS Fargate pricing and Amazon EKS documentation. https://aws.amazon.com/fargate/pricing/

[9] Project source code. Climate Intelligence for Heatwave Monitoring and Early Warning. https://github.com/mitm2006/heat_forcast

[10] Project demonstration video. https://drive.google.com/file/d/1qMQqGe6TYQX5faV7ziA9zck1zI-yx7
view?usp=sharing